

RESEARCH ARTICLE

Segment-Based May-Happen-in-Parallel Analysis for C Programs

Faming Lu¹ | Xiaoyu Wang¹  | Qingtian Zeng¹ | Guiyuan Yuan¹ | Yunxia Bao²

¹College of Computer Science and Engineering, Shandong University of Science and Technology, Qingdao, China | ²College of Mathematics and System Science, Shandong University of Science and Technology, Qingdao, China

Correspondence: Guiyuan Yuan (gyuan_yuan@163.com) | Yunxia Bao (baoyunxia98@163.com)

Received: 24 November 2024 | **Revised:** 31 May 2025 | **Accepted:** 5 July 2025

Funding: This work was supported by the Natural Science Foundation of Shandong Province under grant nos. ZR2023MF097, ZR2024QF107 and ZR2024ZD22; National Major Science and Technology Projects of China under grant no. 2022ZD0119501; and the National Natural Science Foundation of China under grant no. 52374221.

Keywords: MHP | multi-threading | Pthreads | static analysis

ABSTRACT

May-Happen-in-Parallel (MHP) analysis serves as the basis for many concurrency bugs analyses. Inadequate handling of the coupling between locks and thread creation statements, as well as inter-procedural locks, can lead to a loss of precision in MHP analysis. To address these issues, this paper proposes a new MHP analysis for C that operates at the segment granularity rather than individual statements. By constructing a Segmented Thread-sensitive Control Flow Graph (STCFG) for a program, statements are grouped into different segments. Context information is added to these segments to capture the semantics of Pthreads operations, thereby identifying Happens-Before (HB) and conflict relationships between segments. To compute MHP information for statement pairs, it is sufficient to examine the relationship between segments to infer the relationship between statements. We implement our algorithm in LLVM and evaluate it using eight test cases as well as four programs from the SPLASH2 benchmark suite. Preliminary results show that our method provides higher precision and achieves higher efficiency.

1 | Introduction

The C language plays an important role in many fields, where programs often need to use concurrent structures to enhance performance. It provides developers with flexible concurrency patterns, but these flexible patterns also introduce more concurrency issues, such as data races [1–3] and deadlocks [4, 5]. MHP analysis determines whether two statements in a program can run concurrently and is a crucial part of detecting concurrency bugs like data races. Unfortunately, MHP analysis often struggles to balance speed and precision. For large multithreaded programs, performing a sound and complete analysis of whether any two statements can run concurrently is NP-complete [6].

Many MHP analyses enhance precision by leveraging features specific to the languages they analyze. Specifically, the X10 language provides structured concurrency control through async-finish constructs, which avoid complex thread interleavings. As a result, MHP analysis of X10 programs [7–9] is simpler and achieves higher accuracy in detecting concurrency bugs. In contrast, C features lower-level concurrency control structures and more complex thread scheduling. Figure 1 illustrates several possible complex concurrency patterns: Figure 1a shows that t_2 is fully joined into t_1 . Figure 1b shows that t_2 is not joined into t_1 and t_2 has a longer lifespan than t_1 . Figure 1c shows that t_1 creates two threads t_2 and t_3 , but t_3 is joined into t_3 after complete and t_3 is ultimately joined to t_1 through t_2 . Figure 1d shows that t_2 is

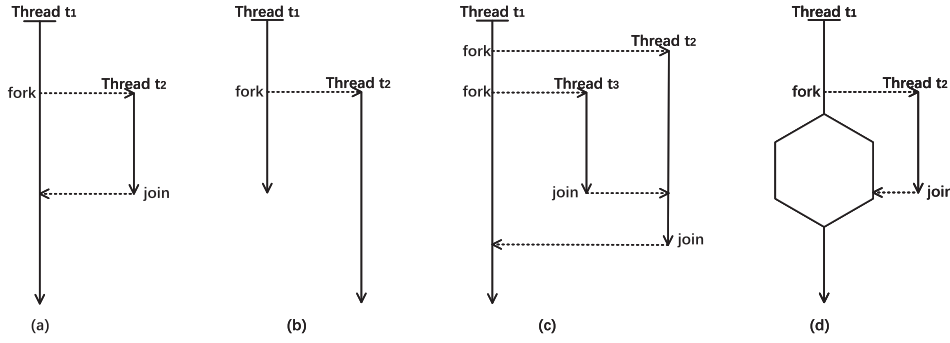


FIGURE 1 | Complex concurrency patterns. (a) Full join. (b) Not join. (c) Indirect join. (d) Partial join.

joined into t_1 in some cases but not in others. These complex relationships make MHP analyses designed for structured languages unsuitable for C. Thus, even with the most precise alias analyses [10–12], we still need to deal with interleaving complexity.

For C, C++, and Java, there are many significant achievements in MHP analysis. In early studies, Bristow et al. [13] propose constructing inter-procedural precedence graphs to represent the synchronization order between processes. Naumovich [14] applies MHP analysis to Java using a Parallel Execution Graph (PEG), which combines a Control Flow Graph (CFG) of each class that can potentially become a thread. However, it has certain drawbacks, such as the need to limit the number of threads that may correspond to the same CFG and poor handling of join operations. Li et al. [15] improve on Naumovich’s method, enhancing its scalability. Barik [16] proposes a Thread Creation Tree (TCT) model and develops an effective MHP analysis based on TCT. It maps each thread to its creation statement, handling thread creation and joining statements with finer granularity, but it assumes that each thread runs in parallel with its parent or child threads, which can lead to a large number of false positives. Naik et al. [17] introduce a flow-sensitive and context-sensitive analysis, but it does not consider join statements. Joisha et al. [18] propose a coarse-grained MHP analysis based on the Procedural Concurrency Graph (PCG), which only computes MHP relations at procedure granularity instead of at statement. Di et al. [19] introduce Thread-sensitive Control Flow Graph (TCFG), which adds fork and join edges to an Interprocedural Control Flow Graph (ICFG), and Region Relation Graph (RRG) models for program analysis. They perform flow-sensitive HB analysis on a TCFG and propose a region-based MHP analysis. Zhou et al. [20] propose a static vector clock-based MHP analysis, incorporating static vector clocks into MHP analysis.

However, even the latest MHP analyses only examine whether there is an HB relationship between statements caused by thread creation or join operations during HB analysis. This actually overlooks coupling information between locks and thread creation operations, leading to a missing HB relationship and reduced analysis precision. Moreover, in terms of conflict analysis, existing methods may introduce inaccuracies in MHP analysis due to incorrect handling of inter-procedural locks. These are limitations that this paper aims to overcome.

This paper proposes a new MHP analysis for C programs. In terms of HB relationship analysis, we not only consider the

semantics of thread creation and join statements but also the coupling between locks and thread creation statements to improve HB analysis precision. In conflict analysis, we not only consider intra-procedural locks but also perform a more detailed analysis of inter-procedural locks, making conflict relationship analysis more precise. Additionally, we introduce STCFG, which segments a program into sections by grouping statements with the same context information together. During MHP analysis, we only need to check whether segments can run concurrently, thereby detecting concurrency relationships within segments and improving analysis efficiency.

Our contributions are summarized as follows:

- A new MHP analysis is proposed that improves HB relationship analysis precision by considering coupling between locks and thread creation statements, and by incorporating more context information during thread creation and joining. The method also enhances conflict relationship analysis precision through a more detailed analysis of inter-procedural locks.
- An STCFG model is introduced, along with its construction algorithm. It segments statements based on concurrency-related context information into different segments. During MHP analysis, segments are used instead of individual statements for HB and conflict relationships analysis.
- The proposed algorithm has been implemented in LLVM 16.0.0 and tested with sample cases and programs from SPLASH2. The results confirm the effectiveness of our algorithm.

2 | Running Example and Motivation

2.1 | Running Example

Figure 2a shows a multithreaded C program with a main thread t_m , child threads t_1 and t_2 [2], lock l , and global variable $gvar$. The program modifies $gvar$ a total of six times, labeled $gvar1$ – $gvar6$.

For convenience, we use *fork* to represent a `pthread_create()` statement, *join* to represent a `pthread_join()` statement, *lock* to represent a `pthread_mutex_lock()` statement, and *unlock* to represent a `pthread_mutex_unlock()` statement. Other statements are represented using *s* with subscripts.

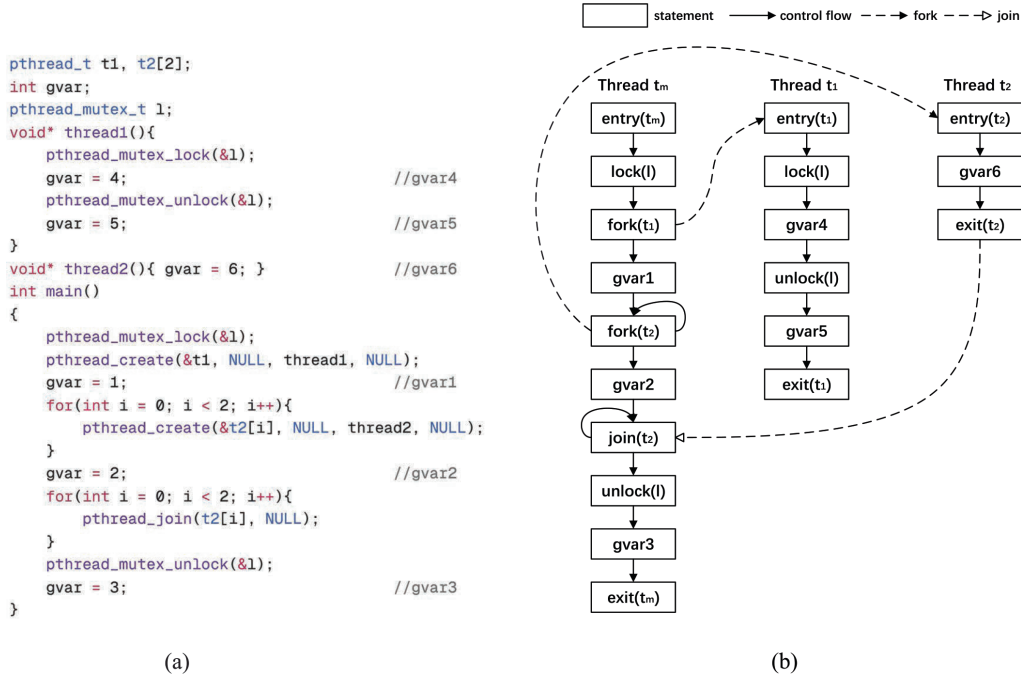


FIGURE 2 | An example and its TCFG. (a) Code. (b) TCFG.

The analysis focuses on the pairs of modifications made by different threads. Since s_{gvar1} and s_{gvar2} are protected by a lock l , they cannot be concurrent with s_{gvar4} . s_{gvar1} and s_{gvar6} have an HB relationship due to *fork*, so they cannot be concurrent.

Although there is no lock protection or HB relationship between s_{gvar1} , s_{gvar2} , and s_{gvar5} due to *fork* or *join*, s_{gvar1} and s_{gvar2} must execute before s_{gvar5} because t_m holds lock l when t_2 starts, causing t_2 to wait until t_m releases l . Therefore, there is an HB relationship due to *lock* and *fork* coupling between s_{gvar1} , s_{gvar2} , and s_{gvar5} , preventing their concurrency. Similarly, $join(t_2)$ and s_{gvar2} must execute before t_1 starts, meaning t_2 must be joined into t_m before t_1 starts executing. Thus, statements in t_2 execute before those in t_1 . Since t_2 's creation statement is in a loop, t_2 corresponds to multiple runtime threads. Therefore, there is no HB relationship among statements within t_2 . As a result, s_{gvar6} , which is not protected by a lock, is concurrent with itself.

Consequently, there are only four pairs of concurrent modifications in the program: s_{gvar2} with s_{gvar6} , s_{gvar3} with s_{gvar4} , s_{gvar3} with s_{gvar5} , and s_{gvar6} with itself.

2.2 | Motivation

Figure 2b presents the TCFG for the program shown in Figure 2a. Its nodes represent program statements. t represents an abstract thread¹, and t_m denotes an abstract thread corresponding to its main thread. Each abstract thread corresponds to a thread creation statement in a program. Typically, each abstract thread corresponds to a single runtime thread. However, in some special cases, an abstract thread may correspond to multiple runtime threads, referred to as a multi-forked thread. To accurately analyze multi-forked threads, we provide a more rigorous definition of such threads.

Definition 1 (Multi-Forked Threads). A thread t is a multi-forked thread if at least one of the following conditions holds:

- Its parent thread is a multi-forked thread;
- If t corresponds to a thread creation statement $fork(t)$, there exists a path P starting from $fork(t)$ and returning to $fork(t)$ such that P does not contain a $join(t)$ statement.

By using a TCFG for MHP analysis, we can identify the HB relationship caused by *fork* or *join*. For instance, there is an HB relationship between s_{gvar1} and s_{gvar6} due to $fork(t_2)$, and an HB relationship between s_{gvar3} and s_{gvar6} due to $join(t_2)$. By performing lock analysis in TCFG, we can also identify conflict relationships. For example, there is a conflict between s_{gvar1} , s_{gvar2} , and s_{gvar4} due to lock l .

However, previous methods assumed that all statements within t_2 are protected by lock l , leading to incorrect conclusions about conflicts. Consequently, s_{gvar2} and s_{gvar6} are mistakenly considered to be in conflict due to l . Statements without HB and conflict relationships are all considered to be concurrent.

Table 1 presents MHP analysis results by using TCFG. Black check marks indicate correctly identified concurrent statement pairs, red check marks indicate pairs that TCFG incorrectly considers concurrent but are not actually concurrent, and green check marks denote pairs that TCFG incorrectly considers non-concurrent but are actually concurrent.

Comparing this with the analysis in Section 2.1, it is evident that errors stem from previous methods not accounting for the coupling between *lock* and *fork*, which can introduce HB relationship between statements, as well as from the incorrect handling of

inter-procedural locks. To address these issues with greater precision in handling context information and inter-procedural locks, we propose the STCFG model.

3 | Segmented Thread-Sensitive Control Flow Graph

To address these limitations, we propose an STCFG model based on flow-sensitive and context-sensitive program analysis. Figure 3 compares TCFG and STCFG representations of program fragments from Figure 2a. In the STCFG, statements are grouped into segments represented by dashed rectangles, where all statements within a segment share unified concurrency properties. By analyzing these properties, we can determine whether there are HB or conflict relationships between segments.

In this section, we first introduce the STCFG construction process. Then, we demonstrate how to utilize segment properties for MHP analysis. Finally, we apply STCFG to analyze the concrete example presented in Figure 2a.

3.1 | Construction of STCFG

Before introducing the STCFG construction algorithm in Section 3.1.4, this section establishes three foundational components: (1) the segment formation mechanism (Section 3.1.1), (2) segment properties (Section 3.1.2), and (3) the initialization and update of properties (Section 3.1.3).

TABLE 1 | MHP statement pairs marked with a checkmark.

	s_{gvar1}	s_{gvar2}	s_{gvar3}	s_{gvar4}	s_{gvar5}	s_{gvar6}
s_{gvar1}					✓	
s_{gvar2}					✓	✓
s_{gvar3}				✓	✓	
s_{gvar4}			✓			
s_{gvar5}	✓	✓	✓			✓
s_{gvar6}		✓			✓	✓

3.1.1 | Segment Formation Mechanism

In an STCFG, a segment is represented as a node, and each segment contains at least one program statement. When segmenting a program, if a statement does not affect the properties of the segment, it is added to the current segment. However, if a statement has the potential to alter the properties of the current segment, a new segment must be created.

Figure 4 illustrates six scenarios in which a new segment should be created during the construction of an STCFG. These scenarios are labeled as A–F. A new segment is needed when entering a new thread, corresponding to scenario A. Encountering a *fork*, *join*, *lock*, or *unlock* statement requires creating a new segment, and these cases are labeled as B, C, D, and E, respectively. If a statement s has multiple predecessor statements and does not belong to any existing segment in the current thread, a new segment should be created, which is scenario F.

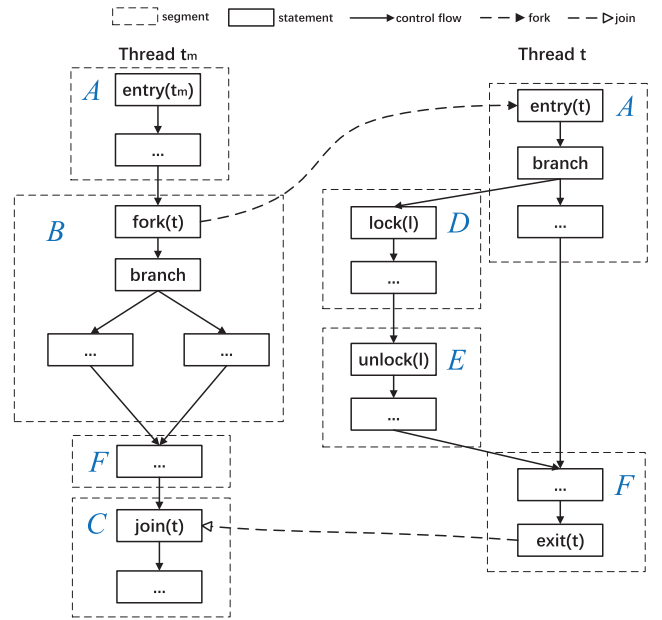


FIGURE 4 | Situations of segment creation.

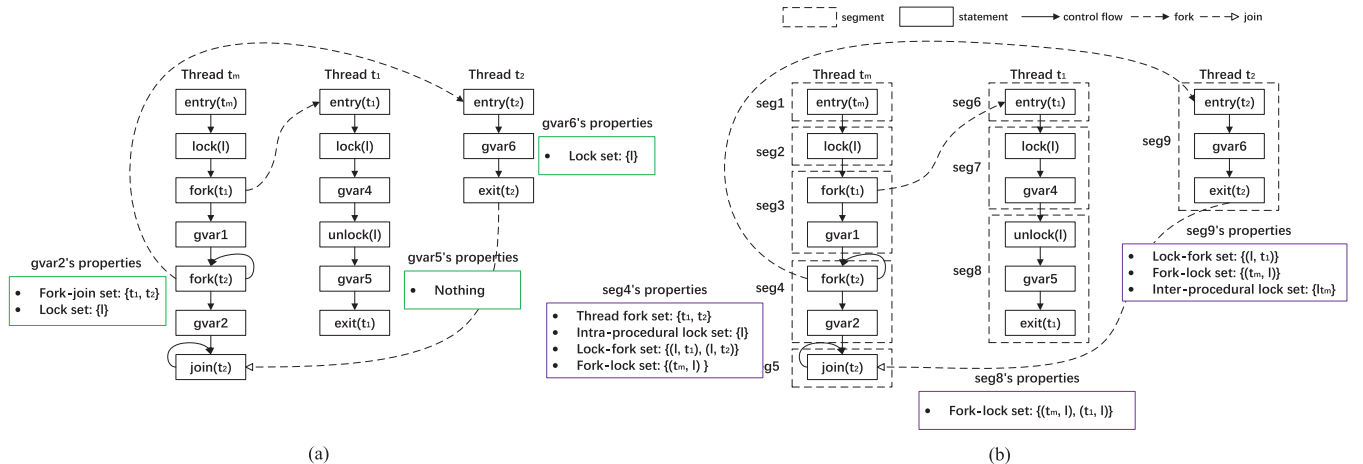


FIGURE 3 | Partial fragments of TCFG and STCFG. (a) TCFG. (b) STCFG.

In Figure 3b, the segmentation follows specific rules. Segments *seg1*, *seg6*, and *seg9* are created when a new thread is initiated. Segments *seg2* and *seg7* are formed when a *lock* statement is encountered. Segments *seg3* and *seg4* result from a *fork* statement. Segment *seg5* is created when a *join* statement is met. Segment *seg8* is established when an *unlock* statement is encountered. In this example, the *fork*(*t2*) statement in *seg4* has two predecessor statements. When analyzed following the control flow, this statement will be analyzed for the second time. However, since it already belongs to an existing segment, no new segment will be created.

3.1.2 | Segment Properties

Upon completing segmentation, each segment initializes and updates its properties. Each segment needs to record the following properties:

1. Thread fork set ($\mathcal{F}$): Records all descendant threads that may execute concurrently.
2. Thread join set ($\mathcal{J}$): Records threads that cannot be concurrent due to *join*.
3. Fork-lock set ($\mathcal{FL}$): Contains thread-lock pairs (t, l), recording locks acquired by a thread after its creation.
4. Lock-fork set ($\mathcal{LF}$): Captures the lock-fork coupling where a thread holds a lock l while creating another thread t , denoted as (l, t).
5. Intra-procedural lock set ($\mathcal{L}$): Records the intra-procedural locks protecting the segment.
6. Inter-procedural lock set ($\mathcal{IL}$): Records inter-procedural locks protecting the segment. An inter-procedural lock, denoted as l_t , where l represents the lock and the subscript t indicates the thread in which the lock is created.

TCFG models HB relationships through four properties (*before fork*, *after join*, *fork-join*, *disjoint program paths*) and uses a lock set to capture conflicts. STCFG offers an enhanced semantic representation through the following aspects: (1) $\mathcal{F}$ and $\mathcal{J}$ that capture fork/join HB relationships, (2) $\mathcal{FL}$ and $\mathcal{LF}$ for modeling the coupling between locks and threads, and (3) Classification of protective locks into intra-procedural and inter-procedural types. This classification is necessary because intra-procedural locks are the locks actually acquired by threads and can affect any other thread. In contrast, inter-procedural locks are not the locks actually acquired by threads. Instead, they

are locks assigned by the ancestor thread to ensure that descendant threads are protected from interference by other threads. However, inter-procedural locks do not affect the ancestor thread or other descendant threads of the ancestor.

As shown in the properties within the green rectangle in Figure 3a, *fork-join* set = $\{t_1, t_2\}$ indicates that this statement executes after t_1 and t_2 are forked and before they are joined. This implies that this statement may execute concurrently with the statements in t_1 and t_2 . The *lock* set = $\{l\}$ means that this statement is protected by lock l .

As illustrated by the properties of *seg4* in Figure 3b, $\mathcal{F} = \{t_1, t_2\}$ suggests that *seg4* may execute concurrently with the segments in t_1 and t_2 . $\mathcal{LF} = \{(l, t_1), (l, t_2)\}$ implies that t_1 and t_2 are created while the lock l is held by their ancestor thread t_m . If t_1 or t_2 attempts to acquire l after starting, it must wait for t_m to release the lock. $\mathcal{FL} = \{(t_m, l)\}$ indicates that t_m acquires l before executing this segment. $\mathcal{L} = \{l\}$ means that *seg4* is protected by the intra-procedural lock l . The property $\mathcal{IL} = \{l_{t_m}\}$ of *seg9* indicates that this segment is protected by the inter-procedural lock l_{t_m} , which is created by t_m .

3.1.3 | Initialization and Update of Properties

We describe the initialization rules for the properties in Table 2. In the table, *seg'* denotes the segment being initialized, and its predecessor segment is denoted as *seg*. $\mathcal{J}_t$ denotes the set of threads that must be joined by thread t , and $\mathcal{DES}_t$ denotes the set of threads whose ancestor is t . To correctly compute $\mathcal{J}_t$ and $\mathcal{DES}_t$, which depend on information about child threads, we first analyze these threads. The first row lists the properties to be initialized, the first column specifies the six segment creation scenarios, and the remaining cells detail the corresponding initialization rules. We detail the initialization process for each scenario below.

When initializing a segment *seg'*, its properties are determined by the creation scenario and whether it has a predecessor segment *seg*. If *seg'* is created during thread entry, which means it has no predecessor, all its properties are initialized as empty sets. For other scenarios where *seg'* has a predecessor *seg*, its properties are first initialized as a copy of *seg*'s properties, and then updated according to the specific creation operation:

- *fork*(t): Add thread t to $\mathcal{F}_{seg'}$. Since the current segment can also execute concurrently with $\mathcal{DES}_t$, also add $\mathcal{DES}_t$ to $\mathcal{F}_{seg'}$. Furthermore, add ordered pairs from $\mathcal{L}_{seg} \times \{t\}$ to $\mathcal{LF}_{seg'}$.

TABLE 2 | Initialization rules of STCFG segment properties.

Statements	$\mathcal{F}_{seg'}$	$\mathcal{J}_{seg'}$	$\mathcal{L}_{seg'}$	$\mathcal{FL}_{seg'}$	$\mathcal{LF}_{seg'}$
<i>entry</i> (t)	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
<i>fork</i> (t)	$\mathcal{F}_{seg} \cup \{t\} \cup \mathcal{DES}_t$	$\mathcal{J}_{seg}$	$\mathcal{L}_{seg}$	$\mathcal{FL}_{seg}$	$(\mathcal{L}_{seg} \times \{t\}) \cup \mathcal{LF}_{seg}$
<i>join</i> (t)	$\mathcal{F}_{seg} - \{t\} - \mathcal{J}_t$	$\mathcal{J}_{seg} \cup \{t\} \cup \mathcal{J}_t$	$\mathcal{L}_{seg}$	$\mathcal{FL}_{seg}$	$\mathcal{LF}_{seg} - (\mathcal{L}_{seg} \times \{t\})$
<i>lock</i> (l)	$\mathcal{F}_{seg}$	$\mathcal{J}_{seg}$	$\mathcal{L}_{seg} \cup \{l\}$	$\mathcal{FL}_{seg} \cup \{(t, l)\}$	$\mathcal{LF}_{seg}$
<i>unlock</i> (l)	$\mathcal{F}_{seg}$	$\mathcal{J}_{seg}$	$\mathcal{L}_{seg} - \{l\}$	$\mathcal{FL}_{seg}$	$\mathcal{LF}_{seg} - (\{l\} \times \mathcal{F}_{seg})$
<i>multi-pre</i>	$\mathcal{F}_{seg}$	$\mathcal{J}_{seg}$	$\mathcal{L}_{seg}$	$\mathcal{FL}_{seg}$	$\mathcal{LF}_{seg}$

- *join(t)*: Threads joined by t can no longer execute concurrently with the current segment. Hence, remove t and J_t from $\mathcal{F}_{seg'}$ and add them to $\mathcal{J}_{seg'}$. Additionally, remove ordered pairs in $\mathcal{L}_{seg} \times \{t\}$ from $\mathcal{LF}_{seg'}$.
- *lock(l)*: Add lock l to $\mathcal{L}_{seg'}$, and add ordered pair (t, l) to $\mathcal{FL}_{seg'}$, where t is the current thread.
- *unlock(l)*: Remove lock l from $\mathcal{L}_{seg'}$. Also, remove ordered pairs in $\{l\} \times \mathcal{F}_{seg}$ from $\mathcal{LF}_{seg'}$.
- Multiple predecessors: No additional initialization is required beyond inheriting from the predecessor segment.

Due to the presence of loops and branch-merge structures in programs, a statement being analyzed might already belong to a segment, denoted as seg . Different predecessor segments can have varying impacts on the properties of seg , so both seg and its successor segments need to be updated. We denote predecessor segments that potentially alter seg 's properties as $'seg$. To compute the updates, we first calculate the impact $\mathcal{P}_\Delta$ of $'seg$ on seg and its successors, defined as the set of property differences:

- $\mathcal{F}_\Delta \Leftarrow \mathcal{F}_{'seg} - \mathcal{F}_{seg}$.
- $\mathcal{J}_\Delta \Leftarrow \mathcal{J}_{seg} - \mathcal{J}_{'seg}$.
- $\mathcal{L}_\Delta \Leftarrow \mathcal{L}_{seg} - \mathcal{L}_{'seg}$.
- $\mathcal{FL}_\Delta \Leftarrow \mathcal{FL}_{seg} - \mathcal{FL}_{'seg}$.
- $\mathcal{LF}_\Delta \Leftarrow \mathcal{LF}_{'seg} - \mathcal{LF}_{seg}$.

To ensure soundness while minimizing unnecessary constraints, our approach implements a selective update strategy. Specifically, we deliberately include more threads in $\mathcal{F}_{seg}$ to prevent false negatives. At the same time, we remove redundant entries from $\mathcal{J}_{seg}$, $\mathcal{L}_{seg}$ and $\mathcal{FL}_{seg}$ to reduce false positives. Note that adding threads to $\mathcal{F}_{seg}$ may introduce new thread-lock pairs in $\mathcal{LF}_{seg}$, which are handled by updating $\mathcal{F}_\Delta$ correspondingly.

Algorithm 1 details the method for the update process: (1) Compute $\mathcal{P}_\Delta$ as defined above and (2) Propagate updates through seg and its successor segments using $\mathcal{P}_\Delta$.

3.1.4 | STCFG Construction Algorithm

The STCFG construction algorithm operates as described in Algorithm 2. Initially, the program is segmented according to the methodology presented in Section 3.1.1. For each thread t , the properties of its segments are initialized sequentially using the initialization rules detailed in Section 3.1.3. When descendant threads of t have not yet been completely analyzed, the properties of segments created through *fork* and *join* cannot be initialized. In such cases, the algorithm prioritizes the complete analysis of all descendant threads before proceeding. When a segment is encountered again after its initial analysis, its properties and those of its successor segments are updated using the method specified in Algorithm 1.

After all threads complete their analysis, to accurately capture the semantics of *fork* and *join* operations and enable more efficient MHP analysis using properties, certain context information

ALGORITHM 1 | Properties update algorithm.

Require: $'seg$ and seg .

```

1:  $W \leftarrow$  a worklist of segments.
2: Compute  $\mathcal{P}_\Delta$ .
3:  $W \cup \{seg\}$ .
4: while  $W \neq \emptyset$  do
5:    $seg \Leftarrow$  a statement removed from  $W$ .
6:    $\mathcal{F}_{seg} \Leftarrow \mathcal{F}_{seg} \cup \mathcal{F}_\Delta$ .
7:    $\mathcal{J}_{seg} \Leftarrow \mathcal{J}_{seg} - \mathcal{J}_\Delta$ .
8:    $\mathcal{L}_{seg} \Leftarrow \mathcal{L}_{seg} - \mathcal{L}_\Delta$ .
9:    $\mathcal{FL}_{seg} \Leftarrow \mathcal{FL}_{seg} - \mathcal{FL}_\Delta$ .
10:   $\mathcal{LF}_{seg} \Leftarrow \mathcal{LF}_{seg} \cup \mathcal{LF}_\Delta$ .
11:  if  $\exists$  property of  $seg$  is changed then
12:    for each  $seg' \in succ(seg)$  do
13:      if  $seg'$  is initialized then  $W \cup \{seg'\}$ .
14:    end for
15:  end if
16: end while

```

ALGORITHM 2 | STCFG construction algorithm.

```

1: Segment the program.
2:  $t \leftarrow$  an abstract thread.
3: Initialize  $seg_{entry}$ .
4:  $W \leftarrow$  a worklist of segments.
5:  $W \cup \{seg_{entry}\}$ .
6: while  $W \neq \emptyset$  do
7:    $seg \Leftarrow$  a statement removed from  $W$ .
8:   for each  $seg' \in succ(seg)$  do
9:     if  $seg'$  is uninitialized then
10:      if ready for initialization then
11:        Initialize  $seg'$ .
12:      else
13:        Process descendant threads.
14:      end if
15:       $W \cup \{seg'\}$ .
16:    else
17:      Update the properties of  $seg'$ 
18:    end if
19:  end for
20: end while
21: After all threads complete their analysis:
22: For each  $seg$  created due to  $fork(t)$ , add  $\mathcal{J}_{seg}$  and  $\mathcal{FL}_{seg}$  to all segments of each thread in  $\mathcal{DES}_t$  and  $t$ .
23: For each  $seg$  created due to  $join(t)$ , add  $\mathcal{LF}_{seg}$  to all segments of each thread in  $\mathcal{J}_t$  and  $t$ .
24: Add  $\mathcal{IL}$  to all segments.

```

needs to be propagated to descendant threads. When a thread t is created, threads joined by any ancestor of t cannot execute concurrently with t . Moreover, if an ancestor of t is blocked due to lock acquisition and cannot execute concurrently with other threads, then t must not be able to execute concurrently with those threads. Therefore, for each seg created due to $fork(t)$, $\mathcal{J}_{seg}$ and $\mathcal{FL}_{seg}$ are added to all segments of each thread in $\mathcal{DES}_t$ and t .

Additionally, when a thread t is joined by another thread, if $join(t)$ exhibits a lock-fork coupling with certain segments in some thread and requires waiting for t to complete execution, then all statements in t must also be prevented from concurrent execution with those segments due to the lock-fork coupling. Therefore, for each seg created due to $join(t)$, $\mathcal{LP}_{seg}$ is added to all segments of each thread in J_t and t . For example, in Figure 3b, thread t_2 is joined in $seg5$, which exhibits lock-fork coupling with $seg7$ and $seg8$. Because $seg5$ must wait for t_2 to complete execution, $seg9$ must execute before $seg7$ and $seg8$. Consequently, the pair (l, t_1) is added to $\mathcal{LP}_{seg9}$. This ensures that MHP analysis correctly identifies that $seg9$ must execute before $seg7$ and $seg8$ due to the lock-fork coupling.

Finally, inter-procedural locks are added to all segments. When a thread t satisfies the condition that its $fork(t)$ operation occurs under the protection of an intra-procedural lock l in parent thread t_p , and l remains held until the corresponding $join(t)$ operation, t is considered to be protected by an inter-procedural lock l_{t_p} . In this case, the inter-procedural lock l_{t_p} is added to all segments of thread t and all threads in $\mathcal{DES}_t$. For example, in Figure 3b, $fork(t_2)$ is protected by an intra-procedural lock l in t_m , and l remains held until the $join(t_2)$ operation. Therefore, the inter-procedural lock l_{t_m} is added to segment $seg9$.

3.2 | Computing MHP Relations

After STCFG construction is completed, the properties recorded in the segments are used to determine if there are any HB or conflict relationships between segments. For clarity, we first provide the definitions of segment relationships.

Definition 2 (Inter-Segment Relationships). Given two segments seg and seg' :

- **HB relationship:** If there is a definite order between seg and seg' , seg and seg' are said to have an HB relationship. If seg must be executed before seg' caused by the control flow of the same thread, it is denoted as $seg < seg'$. If seg must be executed before seg' caused by $fork$, it is denoted as $seg <_{fork} seg'$. If seg must be executed before seg' caused by $join$, it is denoted as $seg <_{join} seg'$. If seg must be executed before seg' , caused by the coupling of $lock$ and $fork$, it is denoted as $seg <_{lock-fork} seg'$.
- **Conflict relationship:** If seg and seg' are not concurrent because of intra-procedural locks, it is referred to as an intra-procedural conflict, denoted as $seg\#_{intra} seg'$. If seg and seg' are not concurrent because of inter-procedural locks, it is referred to as an inter-procedural conflict, denoted as $seg\#_{inter} seg'$.
- **Concurrency:** If seg and seg' are concurrent, it is denoted as $seg || seg'$.

Figure 5 illustrates rules for determining the relationship between segments based on properties. Here, t_{il} denotes the thread in which an inter-procedural lock is created, and $\mathcal{T}_{multi-forked}$ represents the set of all multi-forked threads. The segments seg and seg' are the two segments under analysis.

$$\begin{aligned}
[\#_{intra}] \quad & \frac{\mathcal{L}_{seg} \cap \mathcal{L}_{seg'} \neq \emptyset}{seg \#_{intra} seg'} \\
[\#_{inter}] \quad & \frac{\mathcal{IL}_{seg} \cap \mathcal{IL}_{seg'} = \mathcal{IL}_{\cap}, \exists il \in \mathcal{IL}_{\cap} \wedge ((t_{seg} \notin \mathcal{DES}_{t_{il}}) \vee (t_{seg'} \notin \mathcal{DES}_{t_{il}}))}{seg \#_{inter} seg'} \\
[\#_{inter}] \quad & \frac{\mathcal{L}_{seg} \cap \mathcal{IL}_{seg'} \neq \emptyset, t_{seg'} \notin \mathcal{DES}_{t_{seg}}}{seg \#_{inter} seg'} \\
[<] \quad & \frac{t_{seg} = t_{seg'}, t_{seg} \notin \mathcal{T}_{multi-forked}}{seg < seg' \text{ or } seg' < seg} \\
[<_{join}] \quad & \frac{t_{seg} \in \mathcal{J}_{seg'}, t_{seg'} \notin \mathcal{T}_{multi-forked}}{seg <_{join} seg'} \\
[<_{fork}] \quad & \frac{t_{seg'} \notin \mathcal{F}_{seg}, t_{seg'} \notin \mathcal{J}_{seg}, t_{seg'} \in \mathcal{DES}_{t_{seg}}, t_{seg} \notin \mathcal{T}_{multi-forked}}{seg <_{fork} seg'} \\
[<_{lock-fork}] \quad & \frac{(l, t_{seg'}) \in \mathcal{LF}_{seg}, (t_{seg'}, l) \in \mathcal{FL}_{seg'}}{seg <_{lock-fork} seg'}
\end{aligned}$$

FIGURE 5 | Rules for determining the relationship between segments.

1. For intra-procedural conflict relationship, if there is a non-empty intersection between intra-procedural lock sets of two segments, then these segments are considered to have an intra-procedural conflict relationship.
2. For the inter-procedural conflict relationship, if any of the following conditions hold, then two segments are considered to have an inter-procedural conflict relationship:
 - There exists a non-empty intersection $\mathcal{IL}_{\cap}$ between inter-procedural lock sets of two segments. Additionally, there exist inter-procedural locks il in $\mathcal{IL}_{\cap}$ such that t_{il} is not a common ancestor thread of threads from two segments.
 - If $\mathcal{L}_{seg}$ has a non-empty intersection with $\mathcal{IL}_{seg'}$, and thread associated with seg is not an ancestor thread of thread associated with seg' .
3. For HB relationship caused by control flow, if two segments belong to the same thread and that thread is not a multi-forked thread, then an HB relationship caused by control flow is considered to exist between these segments.
4. For HB relationship caused by $join$, if thread t_{seg} associated with seg is included in $\mathcal{J}_{seg'}$ and $t_{seg'}$ is not a multi-forked thread, then an HB relationship of $seg <_{join} seg'$ is considered to exist between seg and seg' .
5. For HB relationship caused by $fork$, if thread $t_{seg'}$ associated with seg' is neither included in $\mathcal{F}_{seg}$ nor $\mathcal{J}_{seg}$, but is included in $\mathcal{DES}_{t_{seg}}$, and t_{seg} is not a multi-forked thread, then an HB relationship of $seg <_{fork} seg'$ is considered to exist between seg and seg' .
6. For HB relationship caused by the coupling of $lock$ and $fork$, if there exists an element of form $(l, t_{seg'})$ in $\mathcal{LF}_{seg}$ and an element of form $(t_{seg'}, l)$ in $\mathcal{FL}_{seg'}$, then an HB relationship of $seg <_{lock-fork} seg'$ is considered to exist between seg and seg' .

If seg and seg' do not have any conflict relationship or HB relationship, they are considered to be concurrent.

3.3 | Example

Applying our method to the program in Figures 2a, 6a presents the STCFG constructed through program analysis. Moreover, Figure 6b details properties of each segment, which are utilized to analyze the HB and conflict relationships between segments.

According to the rules established in Section 3.2, we can compute the relationships between segments. For example, since the intersection of $\mathcal{L}_{seg8}$ and $\mathcal{IL}_{seg10}$ is non-empty, and t_{seg10} is not an ancestor thread of t_{seg8} , $seg8 \#_{inter} seg10$. Additionally, since $(l, t_1) \in \mathcal{PL}_{seg3}$ and $(l, t_1) \in \mathcal{PL}_{seg4}$, and owing to the propagation mechanism of $\mathcal{PL}$ in Algorithm 2, we have $(l, t_1) \in \mathcal{PL}_{seg10}$. Moreover, as $(t_1, l) \in \mathcal{LP}_{seg9}$, it follows that $seg3, seg4$ and $seg10 <_{lock-fork} seg9$. Furthermore, given the absence of any HB or conflict relationship between $seg4$ and $seg10$, we conclude that $seg4 \parallel seg10$, thereby preventing potential false negatives that might arise from confusing intra-procedural and inter-procedural locks.

Figure 6c presents the results of MHP analysis. Through better identification of complex relationships between threads and locks, our MHP analysis reveals 4 pairs involving modifications to global variables, consistent with the analysis in Section 2.1.

4 | Experimental Evaluation

We implement our MHP analysis algorithm using SVF [21, 22] in LLVM (version 16.0.0), employing Andersen's algorithm [23] during pointer analysis phase. To evaluate the effectiveness of our method, we reproduce the TCT and RRG methods and compare them with our method through 12 C programs: Four from the SPLASH2 benchmark suite and eight that we develop ourselves. When compiling the LLVM intermediate representation for test programs, we applied O1-level optimization. All experiments are conducted on a machine with a 2.1 GHz Intel(R) Core(TM) i7-12700 processor, 64 GB of memory, and running Ubuntu Linux (kernel version 5.15.0).

4.1 | Precision

Table 3 presents the experimental results in different programs, including program descriptions, actual data race counts, and the numbers of data races detected by each method along with their true positive counts. Our method achieves better results than both baselines, as reflected by lower false positive and false negative rates. These improvements stem from differences in how these methods handle thread concurrency. TCT struggles to accurately model complex fork/join dependencies. RRG considers only one pair of threads at a time when modeling TCFG properties, which can result in the loss of contextual information due to complex thread interleavings. Neither approach considers lock-fork coupling nor distinguishes between inter-procedural and intra-procedural locking mechanisms. These analytical limitations cause both methods to generate false positives and false negatives in certain test cases, ultimately restricting their ability to precisely identify data races arising from complex concurrent execution patterns.

It is important to clarify that TCT, RRG, and the proposed method are not actual data race detection methods. Instead, these methods only determine whether two program statements can execute in parallel during runtime, without considering whether the data objects accessed by the analyzed load or store operations are global variables or share the same memory address, nor the path conditions required for such execution orders to actually occur. Consequently, these methods produce numerous false positives on certain test programs. For example, as shown in Table 3 for the DRB004 test case, all MHP analysis methods generate false positives because they treat the entire array as a single operation object without determining whether the modified or accessed array elements might reside at the same memory address. Similarly, for the SPLASH2 benchmark programs reported in Table 3, all false positives stem from the lack of consideration regarding whether the accessed data objects are global variables or share the same memory address.

4.2 | Efficiency

In this section, all reported results focus solely on model construction and MHP analysis time, excluding the time required for constructing the SVF intermediate representation and pointer analysis.

Figure 7 compares the execution time of TCT, RRG, and STCFG across four test programs. STCFG consistently outperforms both TCT and RRG, achieving an average execution time of only 21.96% of TCT's and 33.3% faster than RRG. Figure 8 presents the breakdown of model construction and MHP analysis time for the three methods across the four test programs. In the model construction phase, the time consumption of STCFG accounts for only 6.05% of the time required by TCT and achieves a 73.29% reduction compared to RRG. This significant speedup stems from first segmenting the program during the construction of the STCFG, which reduces the complexity of property initialization, updating, and context propagation. In the MHP analysis phase, STCFG achieves a 63.28% improvement over TCT and a 13.47% improvement over RRG. This acceleration results from the context propagation performed during the construction phase of STCFG, eliminating the need for reachability search during analysis.

5 | Conclusion

This paper proposes a novel MHP analysis method for C programs. Compared with existing methods, this work makes three refinements. Specifically, it first incorporates lock-fork coupling relationship and fine-grained mutex lock categorization, thus reducing false positives. Second, it provides additional context information during fork and join operations, thereby improving the precision of concurrency relation detection. Third, it employs a segment-based analysis strategy to enhance computational efficiency in MHP detection. However, the proposed method has certain limitations. It currently only supports mutex locks and fork/join operations, without covering other synchronization statements in the Pthreads standard, which we aim to address in future work.

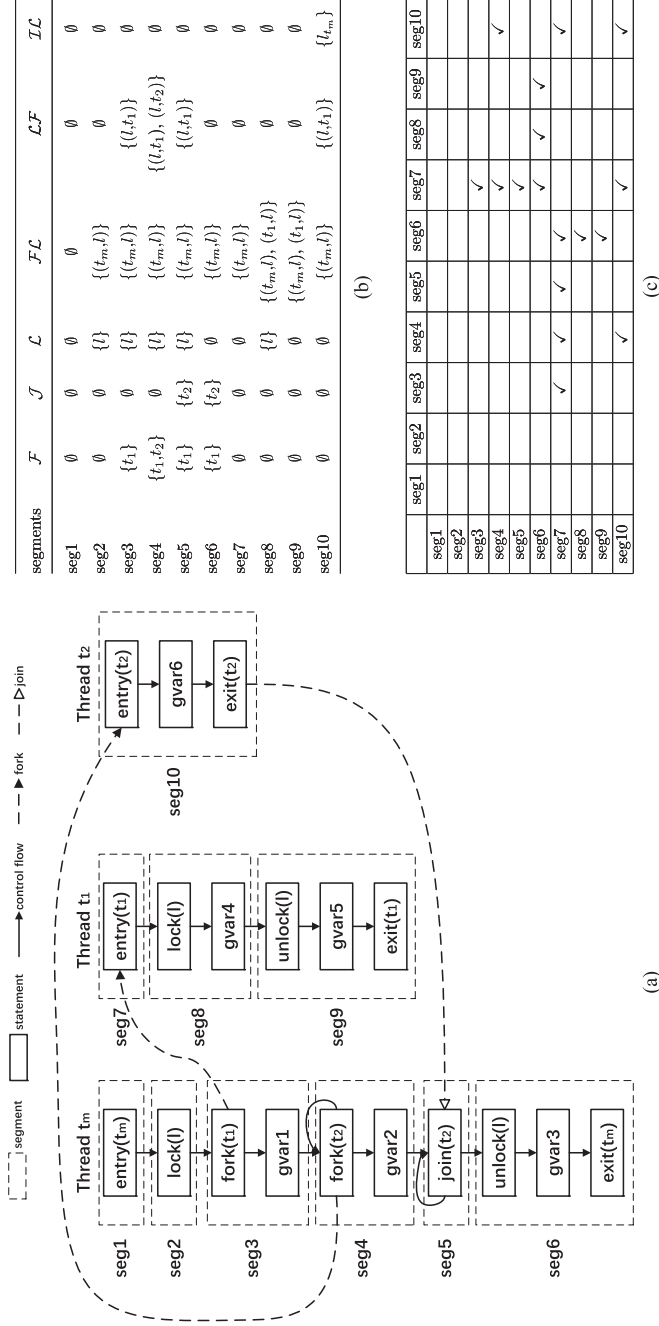
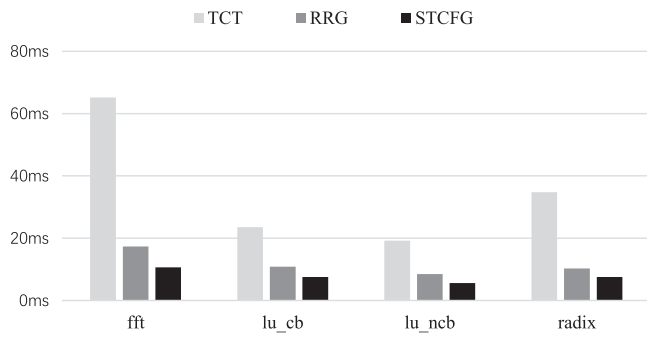
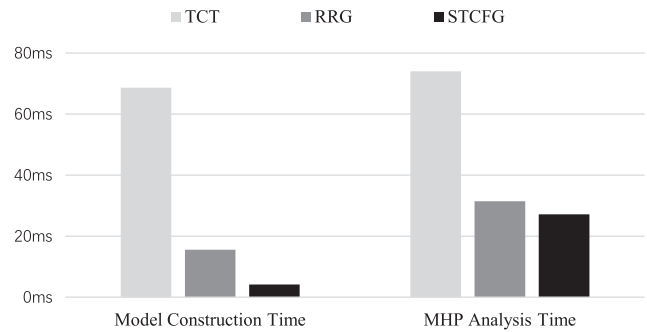


FIGURE 6 | An illustrating example. (a) STCFG. (b) Properties of segments. (c) MHP segment pairs.

TABLE 3 | Data races found by TCT, RRG, and STCFG.

Program	Program introduction	True data race	Number of MHP pairs		
			TCT	RRG	STCFG
DRB001	Simple data race program	2	2/2	2/2	2/2
DRB002	Intra-procedural lock test	0	0/0	0/0	0/0
DRB003	Join test	2	3/2	2/2	2/2
DRB004	Array bounds test	0	3/0	2/0	2/0
DRB005	Program of Figure 2a	4	7/3	6/3	4/4
DRB006	Inter-procedural lock test	9	0/0	0/0	9/9
DRB007	Indirect join test	5	8/5	6/5	5/5
DRB008	Multiple nested join test	9	10/9	9/9	9/9
fft	Complex 1-D FFT	0	803/0	684/0	684/0
lu_cb	Matrix triangulation	0	276/0	169/0	169/0
lu_ncb	Matrix triangulation	0	276/0	169/0	169/0
radix	Integer sort	0	252/0	205/0	205/0

**FIGURE 7** | Execution time comparison of MHP, RRG, and STCFG.**FIGURE 8** | Construction and analysis time comparison of MHP, RRG, and STCFG.

Recent years have witnessed significant advancements in Deep Learning and Large Language Models (LLMs). In particular, LLMs demonstrate strong code understanding through tasks like code generation, completion, and vulnerability repair [24]. However, when tasked with analyzing complex program paths and thread interleavings, LLMs face limitations due to constrained context windows and biases inherent in training data, which result in high false negative rates that impede their practical applicability. To mitigate these challenges, integrating static analysis tools with LLMs has emerged as a promising direction [25, 26]. Static analysis ensures soundness while LLMs excel in array boundary analysis, loop structure interpretation, and path condition reasoning, thereby reducing false positives. Our future

research will explore an MHP analysis framework augmented with LLMs, aiming to simultaneously enhance precision and efficiency while maintaining soundness.

Acknowledgments

We would like to thank the editor, associate editor, and all anonymous reviewers for their valuable and insightful comments, which have helped to improve the quality of the article. This work was supported in part by the National Major Science and Technology Projects of China under grant no. 2022ZD0119501, the National Natural Science Foundation of China under grant no. 52374221, the Natural Science Foundation of Shandong Province of China under grant nos. ZR2023MF097, ZR2024QF107 and ZR2024ZD22.

Data Availability Statement

The data that support the findings of this study are available from the corresponding author upon reasonable request.

Endnotes

¹ The term “thread” means “abstract thread” through the following sections.

References

1. J. W. Voun, R. Jhala, and S. Lerner, “RELAY: Static Race Detection on Millions of Lines of Code,” in *Proceedings of the 6th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering* (Association for Computing Machinery, 2007), 205–214.
2. B. Liu and J. Huang, “D4: Fast Concurrency Debugging With Parallel Differential Analysis,” in *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, vol. 2018 (PLDI, 2018), 359–373.
3. S. Blackshear, N. Gorogiannis, P. W. O’Hearn, and I. Sergey, “RacerD: Compositional Static Race Detection,” in *Proceedings of the ACM on Programming Languages*, vol. 2 (ACM, OOPSLA, 2018), 1–28.
4. B. Liu, P. Liu, Y. Li, C. C. Tsai, D. Da Silva, and J. Huang, “When Threads Meet Events: Efficient and Precise Static Race Detection With Origins,” in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, vol. 2021 (PLDI, 2021), 725–739.
5. Y. Cai, P. Yao, C. Ye, and C. Zhang, “Place Your Locks Well: Understanding and Detecting Lock Misuse Bugs,” in *Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23)* (USENIX Association, 2023), 3727–3744.
6. R. N. Taylor, “A General-Purpose Algorithm for Analyzing Concurrent Programs,” *Communications of the ACM* 26, no. 5 (1983): 361–376.
7. S. Agarwal, R. Barik, V. Sarkar, and R. K. Shyamasundar, “May-Happen-In-Parallel Analysis of X10 Programs,” in *Proceedings of the 12th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Association for Computing Machinery, 2007), 183–193.
8. A. Sankar, S. Chakraborty, and V. K. Nandivada, “Improved MHP Analysis,” in *Proceedings of the 25th International Conference on Compiler Construction* (Association for Computing Machinery, 2016), 207–217.
9. S. Saha and V. K. Nandivada, “On the Fly MHP Analysis,” in *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Association for Computing Machinery, 2020), 173–186.
10. Y. Sui and J. Xue, “On-Demand Strong Update Analysis via Value-Flow Refinement,” in *Proceedings of the 2016 24th ACM SIGSOFT*

11. T. Tan, Y. Li, and J. Xue, “Efficient and Precise Points-To Analysis: Modeling the Heap by Merging Equivalent Automata,” in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Association for Computing Machinery, 2017), 278–291.
12. Y. Li, T. Tan, A. Möller, and Y. Smaragdakis, “A Principled Approach to Selective Context Sensitivity for Pointer Analysis,” *ACM Transactions on Programming Languages and Systems (TOPLAS)* 42, no. 2 (2020): 1–40.
13. G. Bristow, C. Drey, B. Edwards, and W. Riddle, “Anomaly Detection in Concurrent Programs,” in *Proceedings of the 4th International Conference on Software Engineering* (IEEE Press, 1979), 265–273.
14. G. Naumovich, G. S. Avrunin, and L. A. Clarke, “An Efficient Algorithm for Computing MHP Information for Concurrent Java Programs,” in *Software Engineering—ESEC/FSE’99* (Association for Computing Machinery, 1999).
15. L. Li and C. Verbrugge, “A Practical MHP Information Analysis for Concurrent Java Programs,” in *International Workshop on Languages and Compilers for Parallel Computing* (Springer, 2004), 194–208.
16. R. Barik, “Efficient Computation of May-Happen-In-Parallel Information for Concurrent Java Programs,” in *International Workshop on Languages and Compilers for Parallel Computing* (Springer, 2005), 152–169.
17. M. Naik, A. Aiken, and J. Whaley, “Effective Static Race Detection for Java,” in *Proceedings of the 27th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Association for Computing Machinery, 2006), 308–319.
18. P. G. Joisha, R. S. Schreiber, P. Banerjee, H. J. Boehm, and D. R. Chakrabarti, “A Technique for the Effective and Automatic Reuse of Classical Compiler Optimizations on Multithreaded Code,” in *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Association for Computing Machinery, 2011), 623–636.
19. P. Di, Y. Sui, D. Ye, and J. Xue, “Region-Based May-Happen-In-Parallel Analysis for C Programs,” in *Proceedings of the 2015 44th International Conference on Parallel Processing* (IEEE, 2015), 889–898.
20. Q. Zhou, L. Li, L. Wang, J. Xue, and X. Feng, “May-Happen-In-Parallel Analysis With Static Vector Clocks,” in *Proceedings of the 2018 International Symposium on Code Generation and Optimization* (Association for Computing Machinery, 2018), 228–240.
21. Y. Sui, D. Ye, and J. Xue, “Detecting Memory Leaks Statically With Full-Sparse Value-Flow Analysis,” *IEEE Transactions on Software Engineering* 40, no. 2 (2014): 107–122.
22. Y. Sui and J. Xue, “SVF: Interprocedural Static Value-Flow Analysis in LLVM,” in *Proceedings of the 25th International Conference on Compiler Construction* (ACM, 2016), 265–266.
23. L. O. Andersen, “Program Analysis and Specialization for the C Programming Language,” 1994.
24. X. Zhou, S. Cao, X. Sun, and D. Lo, “Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead,” in *ACM Transactions on Software Engineering and Methodology* (Association for Computing Machinery, 2024).
25. H. Li, Y. Hao, Y. Zhai, and Z. Qian, “Enhancing Static Analysis for Practical Bug Detection: An Llm-Integrated Approach,” *Proceedings of the ACM on Programming Languages* 8, no. OOPSLA1 (2024): 474–499.
26. Z. Li, S. Dutta, and M. Naik, “IRIS: LLM-assisted static analysis for detecting security vulnerabilities[J],” *ArXiv Preprint ArXiv:2405.17238* (2024).